dsp.RLSFilter System object

Package: dsp

Compute output, error and coefficients using Recursive Least Squares (RLS) algorithm

Description

The `RLSFilter` object filters each channel of the input using RLS filter implementations.

To filter each channel of the input:

1. Define and set up your RLS filter. See Construction.
2. Call `step` to filter each channel of the input according to the properties of `dsp.RLSFilter`. The behavior of `step` is specific to each object in the toolbox.

Note: Starting in R2016b, instead of using the `step` method to perform the operation defined by the System object™, you can call the object with arguments, as if it were a function. For example, `y = step(obj,x)` and `y = obj(x)` perform equivalent operations.

Construction

`rlsFilt = dsp.RLSFilter` returns an adaptive RLS filter System object, `rlsFilt`. This System object computes the filtered output, filter error and the filter weights for a given input and desired signal using the RLS algorithm.

`rlsFilt = dsp.RLSFilter('PropertyName',PropertyValue, ...)` returns an RLS filter System object, `rlsFilt`, with each specified property set to the specified value.

`rlsFilt = dsp.RLSFilter(LEN, 'PropertyName', PropertyValue, ...)` returns an RLS filter System object, `rlsFilt`. This System object has the Length property set to `LEN`, and other specified properties set to the specified values.

Properties

`Method`	Method to calculate the filter coefficients You can specify the method used to calculate filter coefficients as one of | `Conventional RLS` [1] [2] | `Householder RLS` [3] [4] | `Sliding-window RLS` [5][1][2] | `Householder sliding-window RLS` [4] | `QR decomposition` [1] [2]. The default value is `Conventional RLS`. This property is nontunable.
`Length`	Length of filter coefficients vector Specify the length of the RLS filter coefficients vector as a scalar positive integer value. The default value is `32`. This property is nontunable.
`SlidingWindowBlockLength`	Width of the sliding window Specify the width of the sliding window as a scalar positive integer value greater than or equal to the Length property value. This property is applicable only when the Method property is set to `Sliding-window RLS` or `Householder sliding-window RLS`. The default value is `48`. This property is nontunable.
`ForgettingFactor`	RLS forgetting factor Specify the RLS forgetting factor as a scalar positive numeric value less

	than or equal to 1. Setting this property value to 1 denotes infinite memory, while adapting to find the new filter. The default value is 1. This property is tunable.
`InitialCoefficients`	Initial coefficients of the filter Specify the initial values of the FIR adaptive filter coefficients as a scalar or a vector of length equal to the `Length` property value. The default value is `0`. This property is tunable.
`InitialInverseCovariance`	Initial inverse covariance Specify the initial values of the inverse covariance matrix of the input signal. This property must be either a scalar or a square matrix, with each dimension equal to the `Length` property value. If you set a scalar value, the `InverseCovariance` property is initialized to a diagonal matrix with diagonal elements equal to that scalar value. This property applies only when the `Method` property is set to `Conventional RLS` or `Sliding-window RLS`. The default value is `1000`. This property is tunable.
`InitialSquareRootInverseCovariance`	Initial square root inverse covariance Specify the initial values of the square root inverse covariance matrix of the input signal. This property must be either a scalar or a square matrix with each dimension equal to the `Length` property value. If you set a scalar value, the `SquareRootInverseCovariance` property is initialized to a diagonal matrix with diagonal elements equal to that scalar value. This property applies only when the `Method` property is set to `Householder RLS` or `Householder sliding-window RLS`. The default value is `sqrt(1000)`. This property is tunable.
`InitialSquareRootCovariance`	Initial square root covariance Specify the initial values of the square root covariance matrix of the input signal. This property must be either a scalar or a square matrix with each dimension equal to the `Length` property value. If you set a scalar value, the `SquareRootCovariance` property is initialized to a diagonal matrix with diagonal elements equal to the scalar value. This property applies only when the `Method` property is set to `QR-decomposition RLS`. The default value is `sqrt(1/1000)`. This property is tunable.
`LockCoefficients`	Lock coefficient updates Specify whether the filter coefficient values should be locked. When you set this property to `true`, the filter coefficients are not updated and their values remain the same. The default value is `false` (filter coefficients continuously updated). This property is tunable.

Methods

msesim	Mean-square error for RLS filter
reset	Reset the internal states of a System object
step	Process inputs using RLS filter

Common to All System Objects	
`clone`	Create System object with same property values
`getNumInputs`	Expected number of inputs to a System object
`getNumOutputs`	Expected number of outputs of a System object
`isLocked`	Check locked states of a System object (logical)

`release`	Allow System object property value changes

Examples

expand all

System Identification of an FIR Filter

Algorithms

The `dsp.RLSFilter` System object, when `Conventional RLS` is selected, recursively computes the least squares estimate (RLS) of the FIR filter weights. The System object estimates the filter weights or coefficients, needed to convert the input signal into the desired signal. The input signal can be a scalar or a column vector. The desired signal must have the same data type, complexity, and dimensions as the input signal. The corresponding RLS filter is expressed in matrix form as **P**(*n*) :

$$\mathbf{k}(n) = \frac{\lambda^{-1}\mathbf{P}(n-1)\mathbf{u}(n)}{1+\lambda^{-1}\mathbf{u}^H(n)\mathbf{P}(n-1)\mathbf{u}(n)}$$

$$y(n) = \mathbf{w}^T(n-1)\mathbf{u}(n)$$

$$e(n) = d(n) - y(n)$$

$$\mathbf{w}(n) = \mathbf{w}(n-1) + \mathbf{k}(n)e(n)$$

$$\mathbf{P}(n) = \lambda^{-1}\mathbf{P}(n-1) - \lambda^{-1}\mathbf{k}(n)\mathbf{u}^H(n)\mathbf{P}(n-1)$$

where λ^{-1} denotes the reciprocal of the exponential weighting factor. The variables are as follows:

Variable	Description
n	The current time index
u(*n*)	The vector of buffered input samples at step *n*
P(*n*)	The inverse correlation matrix at step *n*
k(*n*)	The gain vector at step *n*
w(*n*)	The vector of filter tap estimates at step *n*
y(n)	The filtered output at step *n*
e(n)	The estimation error at step *n*
d(n)	The desired response at step *n*
λ	The forgetting factor

u, **w**, and **k** are all column vectors.

References

[1] *M Hayes, Statistical Digital Signal Processing and Modeling, New York: Wiley, 1996*

[2] *S. Haykin, Adaptive Filter Theory,* 4th Edition, *Upper Saddle River, NJ: Prentice Hall, 2002*

[3] A.A. Rontogiannis and S. Theodoridis, "Inverse factorization adaptive least-squares algorithms," Signal Processing, vol. 52, no. 1, pp. 35-47, July 1996.

[4] S.C. Douglas, "Numerically-robust O(N^2) RLS algorithms using least-squares prewhitening," Proc. IEEE Int. Conf. on Acoustics, Speech, and Signal Processing, Istanbul, Turkey, vol. I, pp. 412-415, June 2000.

[5] *A. H. Sayed, Fundamentals of Adaptive Filtering, Hoboken, NJ: John Wiley & Sons, 2003*

Extended Capabilities

C/C++ Code Generation

Generate C and C++ code using MATLAB® Coder™.

See Also

Adaptive Noise Cancellation Using RLS Adaptive Filtering | `dsp.AffineProjectionFilter` | `dsp.FIRFilter` | `dsp.LMSFilter`

Introduced in R2013a